

# Datenbankentwurf

Buchtausch-App

# Projektbeschreibung

Die Datenbank wird zur Verwaltung einer Bücher-Leihbörse erstellt.

Benutzer können ihre Bücher anbieten, nach Büchern anderer Benutzer suchen und Leihanfragen stellen. Buch-Verleihende können diese Anfragen annehmen und einen Übergabe- und einen Rückgabe-Timeslot bestimmen. Aus Datenschutzgründen können Leihende nur den Ort des Verleihenden sehen. Der Verleihende gibt aus Datenschutzgründen nur den Übergabe- und Rückgabe-Treffpunkt an.

Jeder Benutzer besitzt ein persönliches Profil mit seinen angebotenen Büchern. Leihanfragen und Leihungen werden mit ihrem aktuellen Status gespeichert. Administratoren können Benutzerkonten und Bücher verwalten sowie unangemessene Inhalte entfernen.

## Implementation

Die Datenbank wurde auf Grundlage des zuvor erstellten ER-Modells in SQL umgesetzt.

Zunächst wird eine bestehende Datenbank gleichen Namens gelöscht und anschließend neu erstellt. Danach werden die einzelnen Tabellen mit ihren Primär- und Fremdschlüsseln angelegt. Die im ER-Modell definierten Beziehungen werden dabei durch entsprechende Fremdschlüssel und Zwischentabellen für N:M-Beziehungen umgesetzt.

Anschließend werden die Tabellen mit Testdaten gefüllt. Jede Tabelle enthält mindestens zehn Datensätze (Ausnahme: User-Rollen). Abschließend werden die in der Anforderung definierten Funktionen durch SQL-Statements getestet, beispielsweise das Anlegen und Ändern von Benutzern, das Suchen und Ausleihen von Büchern sowie das Bewerten von Büchern und Verwalten von Beschwerden.

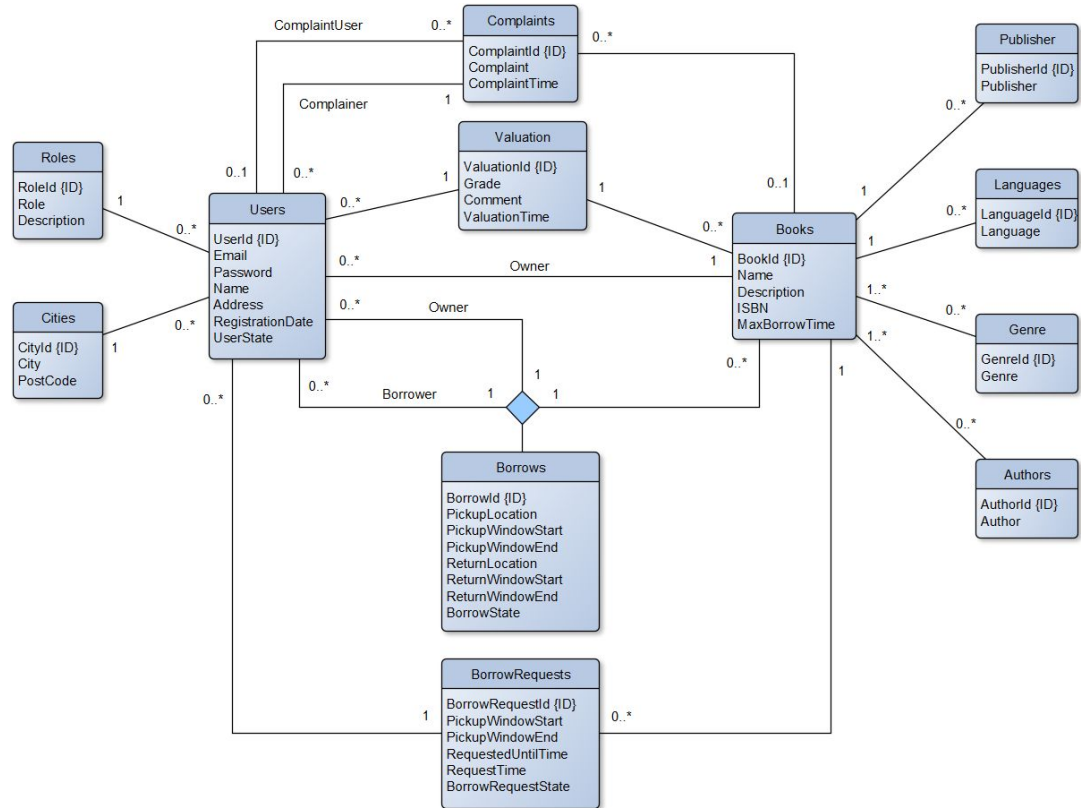
Die Ergebnisse der Testfälle wurden im Datenbankmanagementsystem überprüft und anhand von SQL-Abfragen validiert.

# UML-Diagramm / ER-Modell

Das ER-Modell aus der Konzeptphase wurde ergänzt um die ternäre Beziehung “Borrows” → diese bezieht jetzt neben dem Ausleihenden und dem Buch selbst auch den Besitzer des Buchs mit ein.

Um User sperren zu können, wurde ein “UserState” VARCHAR zur Tabelle “Users” hinzugefügt.

Außerdem wurde auf eigene Tabellen zur Abbildung von Anfrage- und Leih-Status verzichtet. Stattdessen werden diese als VARCHAR abgebildet.



# Setup

```
-- DATENBANK-SETUP

-- Alte Datenbank löschen
DROP DATABASE IF EXISTS bookexchange;

-- Neue Datenbank erstellen
CREATE DATABASE bookexchange;

-- Verwenden der neuen Datenbank
USE bookexchange;
```

Beim Datenbank-Setup wird

1. eine eventuell bereits existierende Datenbank “bookexchange” gelöscht (DROP DATABASE IF EXISTS)
2. anschließend neu angelegt (CREATE)
3. zur Verwendung im weiteren Verlauf des Scripts festgelegt (USE)

✓ MySQL lieferte ein leeres Resultat zurück (d.h. null Datensätze). (Die Abfrage dauerte 0,0003 Sekunden.)

```
-- ----- -- DATENBANK-SETUP -- ----- -- Alte Datenbank löschen DROP DATABASE IF EXISTS
bookexchange;
```

[ Inline bearbeiten ] [ Bearbeiten ] [ PHP-Code erzeugen ]

⚠ Note: #1008 Kann Datenbank 'bookexchange' nicht löschen; Datenbank nicht vorhanden

✓ MySQL lieferte ein leeres Resultat zurück (d.h. null Datensätze). (Die Abfrage dauerte 0,0002 Sekunden.)

```
-- Neue Datenbank erstellen CREATE DATABASE bookexchange CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

[ Inline bearbeiten ] [ Bearbeiten ] [ PHP-Code erzeugen ]

⚠ Error: #1049 Unbekannte Datenbank 'bookexchange'

✓ MySQL lieferte ein leeres Resultat zurück (d.h. null Datensätze). (Die Abfrage dauerte 0,0001 Sekunden.)

```
-- Verwenden der neuen Datenbank USE bookexchange;
```

[ Inline bearbeiten ] [ Bearbeiten ] [ PHP-Code erzeugen ]

Der MySQL-Fehler beim CREATE DATABASE-Befehl ist ein bekannter Logikkonflikt des phpMyAdmin-Import-Assistenten. Da phpMyAdmin beim Datei-Import eine vorausgewählte Standarddatenbank erzwingt, blockiert der Assistent die Ausführung von datenbankübergreifenden Befehlen im Log. Das Skript selbst ist syntaktisch korrekt, löscht und erstellt die Datenbank “bookexchange” wie gewünscht und importiert alle Tabellen erfolgreich.

# Roles / Rollen

```
-- Erstellen der User-Rollen-Tabelle
CREATE TABLE Roles (
  RoleId INTEGER AUTO_INCREMENT PRIMARY KEY,
  Role VARCHAR(50) NOT NULL,
  Description VARCHAR(100)
);

-- Einfügen der 2 Standard-User-Rollen
INSERT INTO Roles (Role, Description)
VALUES
  ('User', 'Kann Bücher suchen und einstellen,
  Leihanfragen stellen, Verleihen, etc.'),
  ('Administrator', 'Verwaltung der
  Datenbank');
```

Die Tabelle “Roles” enthält die möglichen Rollen eines Benutzers. Im aktuellen System sind die Rollen **Admin** und **User** vorgesehen.

Jeder Benutzer erhält genau eine Rolle, während eine Rolle mehreren Benutzern zugeordnet werden kann.

Die “RoleId” dient als Primärschlüssel und wird von der Tabelle Users als Fremdschlüssel referenziert.

## Besondere Attribute

- RoleId → Primärschlüssel

✓ MySQL lieferte ein leeres Resultat zurück (d.h. null Datensätze). (Die Abfrage dauerte 0,0005 Sekunden.)

```
-- ===== -- DATENBANK-TABELLEN ERSTELLEN -- ===== --
Erstellen der User-Rollen-Tabelle CREATE TABLE Roles ( RoleId INTEGER AUTO_INCREMENT PRIMARY KEY, Role
VARCHAR(50) NOT NULL, Description VARCHAR(100) );
```

[\[ Inline bearbeiten \]](#) [\[ Bearbeiten \]](#) [\[ PHP-Code erzeugen \]](#)

✓ 2 Datensätze eingefügt.

ID der eingefügten Zeile: 2 (Die Abfrage dauerte 0,0012 Sekunden.)

```
-- Einfügen der 2 Standard-User-Rollen INSERT INTO Roles (Role, Description) VALUES ('User', 'Kann Bücher
suchen und einstellen, Leihanfragen stellen, Verleihen, etc.'), ('Administrator', 'Verwaltung der
Datenbank');
```

[\[ Inline bearbeiten \]](#) [\[ Bearbeiten \]](#) [\[ PHP-Code erzeugen \]](#)

# Cities / Städte

```
-- Erstellen der Städte-Tabelle
CREATE TABLE Cities (
  CityId INTEGER AUTO_INCREMENT PRIMARY KEY,
  City VARCHAR(200) NOT NULL,
  PostCode VARCHAR(5) NOT NULL
);
```

Die Tabelle “Cities” enthält die Städte und Postleitzahlen der Benutzer.

Durch die separate Speicherung werden wiederholte Angaben vermieden. Mehrere Benutzer können auf denselben Städte-Eintrag verweisen.

Die “CityId” dient als Primärschlüssel und wird von der Tabelle “Users” als Fremdschlüssel verwendet.

## Besondere Attribute

- CityId → Primärschlüssel

✓ MySQL lieferte ein leeres Resultat zurück (d.h. null Datensätze). (Die Abfrage dauerte 0,0003 Sekunden.)

```
-- Erstellen der Städte-Tabelle CREATE TABLE Cities ( CityId INTEGER AUTO_INCREMENT PRIMARY KEY, City VARCHAR(200)
NOT NULL, PostCode VARCHAR(5) NOT NULL );
```

[ [Inline bearbeiten](#) ] [ [Bearbeiten](#) ] [ [PHP-Code erzeugen](#) ]

# Users / Benutzer

```
-- Erstellen der User-Tabelle
CREATE TABLE Users (
  UserId INTEGER AUTO_INCREMENT PRIMARY KEY,
  Email VARCHAR(200) NOT NULL UNIQUE,
  Password VARCHAR(50) NOT NULL,
  Name VARCHAR(200) NOT NULL,
  Address VARCHAR(200),
  CityId INTEGER NOT NULL,
  RoleId INTEGER NOT NULL,
  RegistrationDate DATETIME NOT NULL,
  UserState VARCHAR(30) NOT NULL DEFAULT 'Active',

  FOREIGN KEY (CityId)
    REFERENCES Cities(CityId),

  FOREIGN KEY (RoleId)
    REFERENCES Roles(RoleId)
);
```

Die Tabelle “Users” enthält die Benutzer der Buchtausch-App.

Jedes Buch besitzt genau einen Besitzer, der über “OwnerId” mit der Tabelle “Users” verknüpft ist. Publisher und Sprache werden ebenfalls über Fremdschlüssel referenziert. Die N:M-Beziehungen zu Autoren und Genres werden über separate Zwischentabellen umgesetzt.

## Besondere Attribute

- UserId → Primärschlüssel
- CityId → Fremdschlüssel auf Cities
- RoleId → Fremdschlüssel auf Roles
- UserState → Userstatus ‘Active’ oder ‘Banned’, Defaultwert ist ‘Active’

✓ MySQL lieferte ein leeres Resultat zurück (d.h. null Datensätze). (Die Abfrage dauerte 0,0003 Sekunden.)

```
-- Erstellen der User-Tabelle CREATE TABLE Users ( UserId INTEGER AUTO_INCREMENT PRIMARY KEY, Email VARCHAR(200) NOT
NULL UNIQUE, Password VARCHAR(50) NOT NULL, Name VARCHAR(200) NOT NULL, Address VARCHAR(200), CityId INTEGER NOT
NULL, RoleId INTEGER NOT NULL, RegistrationDate DATETIME NOT NULL, UserState VARCHAR(30) NOT NULL DEFAULT 'Active',
FOREIGN KEY (CityId) REFERENCES Cities(CityId), FOREIGN KEY (RoleId) REFERENCES Roles(RoleId) );
```

[ [Inline bearbeiten](#) ] [ [Bearbeiten](#) ] [ [PHP-Code erzeugen](#) ]

# Authors / Autoren

```
-- Erstellen der Autoren-Tabelle
CREATE TABLE Authors (
  AuthorId INTEGER AUTO_INCREMENT PRIMARY KEY,
  Author VARCHAR(200) NOT NULL
);
```

Die Tabelle “Authors” enthält die Autoren von angebotenen Bücher, damit diese nicht mehrfach in der Tabelle “Books” erstellt werden müssen, und auch Mehrfach-Autoren möglich sind.

Die N:M-Beziehung zu Autoren wird über separate Zwischentabellen umgesetzt.

## Besondere Attribute

- AuthorId → Primärschlüssel

✓ MySQL lieferte ein leeres Resultat zurück (d.h. null Datensätze). (Die Abfrage dauerte 0,0003 Sekunden.)

```
-- Erstellen der Autoren-Tabelle CREATE TABLE Authors ( AuthorId INTEGER AUTO_INCREMENT PRIMARY KEY, Author
VARCHAR(200) NOT NULL );
```

[ [Inline bearbeiten](#) ] [ [Bearbeiten](#) ] [ [PHP-Code erzeugen](#) ]



# Publishers / Verlage

```
-- Erstellen der Publisher-Tabelle
CREATE TABLE Publishers (
  PublisherId INTEGER AUTO_INCREMENT PRIMARY KEY,
  Publisher VARCHAR(200) NOT NULL
);
```

Die Tabelle “Publishers” enthält die Verlage von angebotenen Bücher, damit diese nicht mehrfach in der Tabelle “Books” gespeichert werden.

Mehrere Bücher können auf denselben Publisher-Eintrag verweisen.  
Jedes Buch besitzt genau einen Publisher.

Die “PublisherId” dient als Primärschlüssel und wird von der Tabelle “Books” als Fremdschlüssel verwendet.

## Besondere Attribute

- PublisherId → Primärschlüssel

✓ MySQL lieferte ein leeres Resultat zurück (d.h. null Datensätze). (Die Abfrage dauerte 0,0003 Sekunden.)

```
-- Erstellen der Publisher-Tabelle CREATE TABLE Publishers ( PublisherId INTEGER AUTO_INCREMENT PRIMARY KEY,
Publisher VARCHAR(200) NOT NULL );
```

[\[ Inline bearbeiten \]](#) [\[ Bearbeiten \]](#) [\[ PHP-Code erzeugen \]](#)

# Languages / Sprachen

```
-- Erstellen der Sprachen-Tabelle
CREATE TABLE Languages (
  LanguageId INTEGER AUTO_INCREMENT PRIMARY KEY,
  Language VARCHAR(50) NOT NULL
);
```

Die Tabelle “Languages” enthält die Sprachen von angebotenen Büchern.

Durch die separate Speicherung werden wiederholte Angaben vermieden. Jedes Buch besitzt genau eine Sprache.

Die “LanguageId” dient als Primärschlüssel und wird von der Tabelle “Books” als Fremdschlüssel verwendet.

## Besondere Attribute

- LanguageId → Primärschlüssel

✓ MySQL lieferte ein leeres Resultat zurück (d.h. null Datensätze). (Die Abfrage dauerte 0,0003 Sekunden.)

```
-- Erstellen der Sprachen-Tabelle CREATE TABLE Languages ( LanguageId INTEGER AUTO_INCREMENT PRIMARY KEY, Language
VARCHAR(50) NOT NULL );
```

[ [Inline bearbeiten](#) ] [ [Bearbeiten](#) ] [ [PHP-Code erzeugen](#) ]

# Genres

```
-- Erstellen der Genre-Tabelle
CREATE TABLE Genres (
  GenreId INTEGER AUTO_INCREMENT PRIMARY KEY,
  Genre VARCHAR(100) NOT NULL
);
```

Die Tabelle “Genres” enthält alle möglichen Genres von angebotenen Bücher, damit diese nicht mehrfach in der Tabelle “Books” erstellt werden müssen, und auch Mehrfach-Genres möglich sind.

Die N:M-Beziehungen zu “Books” und “Genres” werden über separate Zwischentabellen umgesetzt.

## Besondere Attribute

- LanguageId → Primärschlüssel

✓ MySQL lieferte ein leeres Resultat zurück (d.h. null Datensätze). (Die Abfrage dauerte 0,0003 Sekunden.)

```
-- Erstellen der Genre-Tabelle CREATE TABLE Genres ( GenreId INTEGER AUTO_INCREMENT PRIMARY KEY, Genre VARCHAR(100)
NOT NULL );
```

[\[ Inline bearbeiten \]](#) [\[ Bearbeiten \]](#) [\[ PHP-Code erzeugen \]](#)

# Books / Bücher

```
-- Erstellen der Book-Tabelle
CREATE TABLE Books (
  BookId INTEGER AUTO_INCREMENT PRIMARY KEY,
  Name VARCHAR(200) NOT NULL,
  Description VARCHAR(500),
  ISBN VARCHAR(17),
  PublisherId INTEGER,
  LanguageId INTEGER,
  MaxBorrowTime INTEGER NOT NULL,
  OwnerId INTEGER NOT NULL,

  FOREIGN KEY (PublisherId)
    REFERENCES Publishers(PublisherId),

  FOREIGN KEY (LanguageId)
    REFERENCES Languages(LanguageId),

  FOREIGN KEY (OwnerId)
    REFERENCES Users(UserId)
    ON DELETE CASCADE
);
```

Die Tabelle “Books” enthält die von Benutzern angebotenen Bücher.

Jedes Buch besitzt genau einen Besitzer, der über “OwnerId” mit der Tabelle “Users” verknüpft ist. Publisher und Sprache werden ebenfalls über Fremdschlüssel referenziert. Die N:M-Beziehungen zu Autoren und Genres werden über separate Zwischentabellen umgesetzt.

Wird der “besitzende” User gelöscht, werden auch seine Bücher gelöscht (ON DELETE CASCADE).

## Besondere Attribute

- BookId → Primärschlüssel
- OwnerId → Fremdschlüssel auf Users
- PublisherId → Fremdschlüssel auf Publishers
- LanguageId → Fremdschlüssel auf Languages
- MaxBorrowTime → maximale Leihdauer in Tagen

✓ MySQL lieferte ein leeres Resultat zurück (d.h. null Datensätze). (Die Abfrage dauerte 0,0004 Sekunden.)

```
-- Erstellen der Book-Tabelle CREATE TABLE Books ( BookId INTEGER AUTO_INCREMENT PRIMARY KEY, Name VARCHAR(200) NOT
NULL, Description VARCHAR(500), ISBN VARCHAR(17), PublisherId INTEGER, LanguageId INTEGER, MaxBorrowTime INTEGER NOT
NULL, OwnerId INTEGER NOT NULL, FOREIGN KEY (PublisherId) REFERENCES Publishers(PublisherId), FOREIGN KEY
(LanguageId) REFERENCES Languages(LanguageId), FOREIGN KEY (OwnerId) REFERENCES Users(UserId) ON DELETE CASCADE );
```

[\[ Inline bearbeiten \]](#) [\[ Bearbeiten \]](#) [\[ PHP-Code erzeugen \]](#)

# BooksAuthors / Autoren der Bücher

-- Erstellen der Book-Autor-Beziehungstabelle

```
CREATE TABLE BookAuthors (  
  BookId INTEGER NOT NULL,  
  AuthorId INTEGER NOT NULL,  
  
  PRIMARY KEY (BookId, AuthorId),  
  
  FOREIGN KEY (BookId)  
    REFERENCES Books(BookId)  
    ON DELETE CASCADE,  
  
  FOREIGN KEY (AuthorId)  
    REFERENCES Authors(AuthorId)  
);
```

Die Tabelle “BooksAuthors” bildet die N:M-Beziehungen von Büchern zu Autoren ab.

Jedes Buch kann mehrere Autoren haben, referenziert dabei auf “BookId” (Tabelle Books) und “AuthorId” (Tabelle Authors).

Wird ein Buch gelöscht, werden auch die jeweiligen Buch-Autor-Beziehungen gelöscht (ON DELETE CASCADE).

## Besondere Attribute

- BookId und AuthorId → Primärschlüssel
- BookId → Fremdschlüssel auf Books
- AuthorId → Fremdschlüssel auf Authors

✓ MySQL lieferte ein leeres Resultat zurück (d.h. null Datensätze). (Die Abfrage dauerte 0,0003 Sekunden.)

```
-- Erstellen der Book-Autor-Beziehungstabelle CREATE TABLE BookAuthors (  
  BookId INTEGER NOT NULL, AuthorId INTEGER  
  NOT NULL, PRIMARY KEY (BookId, AuthorId), FOREIGN KEY (BookId) REFERENCES Books(BookId) ON DELETE CASCADE, FOREIGN  
  KEY (AuthorId) REFERENCES Authors(AuthorId) );
```

[ [Inline bearbeiten](#) ] [ [Bearbeiten](#) ] [ [PHP-Code erzeugen](#) ]

# BooksGenres / Genres der Bücher

-- Erstellen der Book-Genre-Beziehungstabelle

```
CREATE TABLE BookGenres (  
  BookId INTEGER NOT NULL,  
  GenreId INTEGER NOT NULL,  
  
  PRIMARY KEY (BookId, GenreId),  
  
  FOREIGN KEY (BookId)  
    REFERENCES Books(BookId)  
    ON DELETE CASCADE,  
  
  FOREIGN KEY (GenreId)  
    REFERENCES Genres(GenreId)  
);
```

Die Tabelle "BooksGenres" bildet die N:M-Beziehungen von Büchern zu Genres ab.

Jedes Buch kann mehrere Genres haben, referenziert dabei auf "BookId" (Tabelle Books) und "GenreId" (Tabelle Genres).

Wird ein Buch gelöscht, werden auch die jeweiligen Buch-Genre-Beziehungen gelöscht (ON DELETE CASCADE).

## Besondere Attribute

- BookId und GenreId → Primärschlüssel
- BookId → Fremdschlüssel auf Books
- GenreId → Fremdschlüssel auf Genres

✓ MySQL lieferte ein leeres Resultat zurück (d.h. null Datensätze). (Die Abfrage dauerte 0,0003 Sekunden.)

```
-- Erstellen der Book-Genre-Beziehungstabelle CREATE TABLE BookGenres ( BookId INTEGER NOT NULL, GenreId INTEGER NOT  
NULL, PRIMARY KEY (BookId, GenreId), FOREIGN KEY (BookId) REFERENCES Books(BookId) ON DELETE CASCADE, FOREIGN KEY  
(GenreId) REFERENCES Genres(GenreId) );
```

[\[ Inline bearbeiten \]](#) [\[ Bearbeiten \]](#) [\[ PHP-Code erzeugen \]](#)

# BorrowRequests / Leihanfragen

```
-- Erstellen der BorrowRequests-Tabelle
CREATE TABLE BorrowRequests (
    BorrowRequestId INTEGER AUTO_INCREMENT PRIMARY KEY,

    UserId INTEGER NOT NULL,
    BookId INTEGER NOT NULL,

    PickupWindowStart DATETIME NOT NULL,
    PickupWindowEnd DATETIME NOT NULL,
    RequestedUntilTime DATETIME NOT NULL,
    RequestTime DATETIME NOT NULL,

    BorrowRequestState VARCHAR(30) NOT NULL,

    FOREIGN KEY (UserId)
        REFERENCES Users(UserId)
        ON DELETE CASCADE,

    FOREIGN KEY (BookId)
        REFERENCES Books(BookId)
        ON DELETE CASCADE
);
```

Die Tabelle "BorrowRequests" enthält die Ausleihanfragen von Benutzern für angebotene Bücher.

Eine Anfrage verknüpft einen Benutzer mit einem Buch und enthält den gewünschten Zeitraum sowie den Zeitpunkt der Anfrage.

Der Status "BorrowRequestState" gibt an, ob eine Anfrage beispielsweise noch ausstehend, angenommen oder abgelehnt wurde ("Pending", "Accepted", "Rejected"). Eine Anfrage kann anschließend in eine tatsächliche Ausleihe übergehen.

Wird ein Buchbesitzer oder ein Buch gelöscht, werden auch die jeweiligen BorrowRequests gelöscht (ON DELETE CASCADE).

## Besondere Attribute

- BorrowRequestId → Primärschlüssel
- UserId → Fremdschlüssel auf Users
- BookId → Fremdschlüssel auf Books
- PickupWindowStart / PickupWindowEnd → Abholzeitraum
- RequestedUntilTime → auszuleihen bis
- BorrowRequestState → Status der Anfrage

✓ MySQL lieferte ein leeres Resultat zurück (d.h. null Datensätze). (Die Abfrage dauerte 0,0003 Sekunden.)

```
-- Erstellen der BorrowRequests-Tabelle CREATE TABLE BorrowRequests ( BorrowRequestId INTEGER AUTO_INCREMENT PRIMARY
KEY, UserId INTEGER NOT NULL, BookId INTEGER NOT NULL, PickupWindowStart DATETIME NOT NULL, PickupWindowEnd DATETIME
NOT NULL, RequestedUntilTime DATETIME NOT NULL, RequestTime DATETIME NOT NULL, BorrowRequestState VARCHAR(30) NOT
NULL, FOREIGN KEY (UserId) REFERENCES Users(UserId) ON DELETE CASCADE, FOREIGN KEY (BookId) REFERENCES Books(BookId)
ON DELETE CASCADE );
```

[ [Inline bearbeiten](#) ] [ [Bearbeiten](#) ] [ [PHP-Code erzeugen](#) ]

# Borrows / Leihungen

-- Erstellen der Borrows-Tabelle

```
CREATE TABLE Borrows (
  BorrowId INTEGER AUTO_INCREMENT PRIMARY KEY,

  OwnerId INTEGER NOT NULL,
  BorrowerId INTEGER NOT NULL,
  BookId INTEGER NOT NULL,

  PickupLocation VARCHAR(200) NOT NULL,
  PickupWindowStart DATETIME NOT NULL,
  PickupWindowEnd DATETIME NOT NULL,

  ReturnLocation VARCHAR(200),
  ReturnWindowStart DATETIME,
  ReturnWindowEnd DATETIME,

  BorrowState VARCHAR(30) NOT NULL,

  FOREIGN KEY (OwnerId)
    REFERENCES Users(UserId),

  FOREIGN KEY (BorrowerId)
    REFERENCES Users(UserId),

  FOREIGN KEY (BookId)
    REFERENCES Books(BookId)
    ON DELETE CASCADE
);
```

Die Tabelle "Borrows" enthält die tatsächlich durchgeführten Ausleihen. Eine Ausleihe verbindet ein Buch mit seinem Besitzer ("Owner") und dem ausleihenden Benutzer ("Borrower") und bildet damit die Dreifachbeziehung zwischen **Buch, Besitzer und Ausleiher** ab.

Zusätzlich werden die Übergabe- und Rückgabeorte, die jeweiligen Zeitfenster sowie der aktuelle Status der Ausleihe ("Active", "Returned") gespeichert.

Wird ein Buch gelöscht, werden auch die jeweiligen "BorrowRequests" gelöscht (ON DELETE CASCADE).

Um eine Historie der Buch-Leihgaben zu gewährleisten, kann ein User nur endgültig gelöscht werden, wenn zunächst alle Bücher gelöscht werden (und somit auch alle Borrows).

## Besondere Attribute

- BorrowId → Primärschlüssel
- OwnerId → Fremdschlüssel auf Users
- BorrowerId → Fremdschlüssel auf Users
- BookId → Fremdschlüssel auf Books
- PickupLocation → Abholort
- PickupWindowStart / PickupWindowEnd → Abholzeitraum
- ReturnLocation → Rückgabeort
- ReturnWindowStart / ReturnWindowEnd → Rückgabezeitraum
- BorrowState → Status der Anfrage

✓ MySQL lieferte ein leeres Resultat zurück (d.h. null Datensätze). (Die Abfrage dauerte 0,0003 Sekunden.)

```
-- Erstellen der Borrows-Tabelle CREATE TABLE Borrows ( BorrowId INTEGER AUTO_INCREMENT PRIMARY KEY, OwnerId
INTEGER NOT NULL, BorrowerId INTEGER NOT NULL, BookId INTEGER NOT NULL, PickupLocation VARCHAR(200) NOT NULL,
PickupWindowStart DATETIME NOT NULL, PickupWindowEnd DATETIME NOT NULL, ReturnLocation VARCHAR(200),
ReturnWindowStart DATETIME, ReturnWindowEnd DATETIME, BorrowState VARCHAR(30) NOT NULL, FOREIGN KEY (OwnerId)
REFERENCES Users(UserId), FOREIGN KEY (BorrowerId) REFERENCES Users(UserId), FOREIGN KEY (BookId) REFERENCES
Books(BookId) ON DELETE CASCADE );
```

[ Inline bearbeiten ] [ Bearbeiten ] [ PHP-Code erzeugen ]



# Valuations / Bewertungen

```
-- Erstellen der Valuations-Tabelle
CREATE TABLE Valuations (
  ValuationId INTEGER AUTO_INCREMENT PRIMARY KEY,
  BookId INTEGER NOT NULL,
  UserId INTEGER NOT NULL,
  Grade INTEGER NOT NULL,
  Comment VARCHAR(500),
  ValuationTime DATETIME NOT NULL,

  FOREIGN KEY (BookId)
    REFERENCES Books(BookId)
    ON DELETE CASCADE ,

  FOREIGN KEY (UserId)
    REFERENCES Users(UserId)
    ON DELETE CASCADE
);
```

Die Tabelle “Valuations” enthält die Bewertungen von Büchern durch Benutzer.

Eine Bewertung verknüpft einen Benutzer mit einem Buch und enthält eine Bewertung von 1 bis 5 sowie optional einen Kommentar. Zusätzlich wird der Zeitpunkt der Bewertung gespeichert.

Wird ein User oder ein Buch gelöscht, werden auch die jeweiligen Valuations gelöscht (ON DELETE CASCADE).

## Besondere Attribute

- ValuationId → Primärschlüssel
- BookId → Fremdschlüssel auf Books
- UserId → Fremdschlüssel auf Users

✓ MySQL lieferte ein leeres Resultat zurück (d.h. null Datensätze). (Die Abfrage dauerte 0,0003 Sekunden.)

```
-- Erstellen der Valuations-Tabelle CREATE TABLE Valuations ( ValuationId INTEGER AUTO_INCREMENT PRIMARY KEY, BookId
INTEGER NOT NULL, UserId INTEGER NOT NULL, Grade INTEGER NOT NULL, Comment VARCHAR(500), ValuationTime DATETIME NOT
NULL, FOREIGN KEY (BookId) REFERENCES Books(BookId) ON DELETE CASCADE , FOREIGN KEY (UserId) REFERENCES
Users(UserId) ON DELETE CASCADE );
```

[\[ Inline bearbeiten \]](#) [\[ Bearbeiten \]](#) [\[ PHP-Code erzeugen \]](#)

# Complaints / Beschwerden

```
-- Erstellen der Complaints-Tabelle
CREATE TABLE Complaints (
  ComplaintId INTEGER AUTO_INCREMENT PRIMARY KEY,
  Complaint VARCHAR(500) NOT NULL,
  BookId INTEGER,
  ComplaintUserId INTEGER NOT NULL,
  ComplainerUserId INTEGER NOT NULL,
  ComplaintTime DATETIME NOT NULL,

  FOREIGN KEY (BookId)
    REFERENCES Books(BookId)
    ON DELETE CASCADE ,

  FOREIGN KEY (ComplaintUserId)
    REFERENCES Users(UserId)
    ON DELETE CASCADE,

  FOREIGN KEY (ComplainerUserId)
    REFERENCES Users(UserId)
    ON DELETE CASCADE
);
```

Die Tabelle "Complaints" enthält von Benutzern eingereichte Beschwerden.

Eine Beschwerde bezieht sich auf einen beteiligten Benutzer (ComplaintUserId) und auf ein bestimmtes Buch. Zusätzlich wird gespeichert, welcher Benutzer (ComplainerUserId) die Beschwerde eingereicht hat und wann sie erstellt wurde. Dadurch können Beschwerden nachvollzogen und vom Administrator verwaltet werden.

Wird ein User oder ein Buch gelöscht, werden auch die jeweiligen Complaints gelöscht (ON DELETE CASCADE).

## Besondere Attribute

- ComplaintId → Primärschlüssel
- ComplaintUserId → Fremdschlüssel auf Users
- ComplainerUserId → Fremdschlüssel auf Users
- BookId → Fremdschlüssel auf Books

✓ MySQL lieferte ein leeres Resultat zurück (d.h. null Datensätze). (Die Abfrage dauerte 0,0004 Sekunden.)

```
-- Erstellen der Complaints-Tabelle CREATE TABLE Complaints ( ComplaintId INTEGER AUTO_INCREMENT PRIMARY KEY,
Complaint VARCHAR(500) NOT NULL, BookId INTEGER, ComplaintUserId INTEGER NOT NULL, ComplainerUserId INTEGER NOT
NULL, ComplaintTime DATETIME NOT NULL, FOREIGN KEY (BookId) REFERENCES Books(BookId) ON DELETE CASCADE , FOREIGN KEY
(ComplaintUserId) REFERENCES Users(UserId) ON DELETE CASCADE, FOREIGN KEY (ComplainerUserId) REFERENCES
Users(UserId) ON DELETE CASCADE );
```

[ [Inline bearbeiten](#) ] [ [Bearbeiten](#) ] [ [PHP-Code erzeugen](#) ]

# Einfügen von Testdaten I

✓ 10 Datensätze eingefügt.

ID der eingefügten Zeile: 10 (Die Abfrage dauerte 0.0009 Sekunden.)

```
-- ----- Testdaten einfügen ----- -- Einfügen von 10 Beispiel-  
Städten INSERT INTO Cities (City, PostCode) VALUES ('Berlin', '10115'), ('Hamburg', '20095'), ('München',  
'80331'), ('Köln', '50667'), ('Frankfurt', '60311'), ('Stuttgart', '70173'), ('Dresden', '01067'),  
'Leipzig', '04109'), ('Hannover', '30159'), ('Bremen', '28195');
```

[ Inline bearbeiten ] [ Bearbeiten ] [ PHP-Code erzeugen ]

✓ 10 Datensätze eingefügt.

ID der eingefügten Zeile: 10 (Die Abfrage dauerte 0.0005 Sekunden.)

```
-- Einfügen von 10 Beispiel-Usern INSERT INTO Users (Email, Password, Name, Address, CityId, RoleId,  
RegistrationDate, UserState) VALUES ('art.vandelay@vandelay-industries.com', 'password1', 'Art Vandelay',  
'Hauptstraße 12', 1, 1, '2026-01-10 09:15:00', 'Active'), ('h.e.pennypacker@pennypecker-imports.com',  
'password2', 'H.E. Pennypacker', 'Bahnhofstraße 5', 2, 1, '2026-01-12 10:30:00', 'Active'),  
'kel.varnsen@varnsen-inc.com', 'password3', 'Kel Varnsen', 'Gartenstraße 8', 3, 1, '2026-01-15 14:20:00',  
'Active'), ('george.costanza@example.com', 'password4', 'George Costanza', 'Schulstraße 21', 4, 1, '2026-
```

[ Bearbeiten ]

✓ 11 Datensätze eingefügt.

ID der eingefügten Zeile: 11 (Die Abfrage dauerte 0.0019 Sekunden.)

```
-- Einfügen von 10 Beispiel-Autoren INSERT INTO Authors (Author) VALUES ('J.R.R. Tolkien'), ('George  
Orwell'), ('Frank Herbert'), ('J.K. Rowling'), ('Isaac Asimov'), ('Agatha Christie'), ('Stephen King'),  
'Ernest Hemingway'), ('Douglas Adams'), ('Terry Pratchett'), ('Stephen Baxter');
```

[ Inline bearbeiten ] [ Bearbeiten ] [ PHP-Code erzeugen ]

✓ 10 Datensätze eingefügt.

ID der eingefügten Zeile: 10 (Die Abfrage dauerte 0.0003 Sekunden.)

```
-- Einfügen von 10 Beispiel-Publishers INSERT INTO Publishers (Publisher) VALUES ('Penguin Books'), ('Random  
House'), ('HarperCollins'), ('Bloomsbury'), ('Tor Books'), ('Simon & Schuster'), ('Macmillan Publishers'),  
'Oxford University Press'), ('Fischer Verlag'), ('Suhrkamp Verlag');
```

[ Inline bearbeiten ] [ Bearbeiten ] [ PHP-Code erzeugen ]

✓ 10 Datensätze eingefügt.

ID der eingefügten Zeile: 10 (Die Abfrage dauerte 0.0003 Sekunden.)

```
-- Einfügen von 10 Beispiel-Sprachen INSERT INTO Languages (Language) VALUES ('Deutsch'), ('Englisch'),  
'Französisch'), ('Spanisch'), ('Italienisch'), ('Portugiesisch'), ('Niederländisch'), ('Schwedisch'),  
'Polnisch'), ('Japanisch');
```

[ Inline bearbeiten ] [ Bearbeiten ] [ PHP-Code erzeugen ]

✓ 10 Datensätze eingefügt.

ID der eingefügten Zeile: 10 (Die Abfrage dauerte 0.0003 Sekunden.)

```
-- Einfügen von 10 Beispiel-Genres INSERT INTO Genres (Genre) VALUES ('Fantasy'), ('Science-Fiction'),  
'Krimi'), ('Thriller'), ('Horror'), ('Abenteuer'), ('Roman'), ('Komödie'), ('Sachbuch'), ('Drama');
```

[ Inline bearbeiten ] [ Bearbeiten ] [ PHP-Code erzeugen ]

✓ 12 Datensätze eingefügt.

ID der eingefügten Zeile: 12 (Die Abfrage dauerte 0.0003 Sekunden.)

```
-- Einfügen von 10 Beispiel-Büchern INSERT INTO Books (Name, Description, ISBN, PublisherId, LanguageId,  
MaxBorrowTime, OwnerId) VALUES ('Der Bluthund', 'Ein Jack-Reacher-Roman.', '9783734110771', 11, 2, 30, 1),  
'1984', 'Dystopischer Roman über einen totalitären Staat. Ein Science-Fiction-Klassiker.',  
'9780451524935', 2, 2, 14, 2), ('Der Herr der Ringe', 'Epische Fantasy-Trilogie mit Elben, Zwerge, Orks,  
und natürlich den Hobbits.', '9780261102385', 1, 2, 30, 3), ('Dune', 'Ein Antikriegs-Roman auf dem  
Wüstenplaneten Arrakis, in einer düsteren Science-Fiction-Zukunft.', '9780441172719', 5, 2, 21, 4), ('Die
```

[ Bearbeiten ]

# Einfügen von Testdaten II

✓ 12 Datensätze eingefügt.

ID der eingefügten Zeile: 12 (Die Abfrage dauerte 0,0002 Sekunden.)

```
-- Einfügen von 10 Beziehungen von Book -> Autor INSERT INTO BookAuthors (BookId,
AuthorId) VALUES (1, 1), (2, 2), (3, 1), (4, 3), (5, 4), (6, 5), (7, 6), (8, 7),
(9, 9), (10, 10), (11, 10), (11, 11);
```

[ Inline bearbeiten ] [ Bearbeiten ] [ PHP-Code erzeugen ]

✓ 15 Datensätze eingefügt.

ID der eingefügten Zeile: 15 (Die Abfrage dauerte 0,0002 Sekunden.)

```
-- Einfügen von 10 Beziehungen von Book -> Genre INSERT INTO BookGenres (BookId,
GenreId) VALUES (1, 1), (2, 7), (2, 2), (3, 1), (4, 2), (5, 1), (6, 2), (7, 3),
(8, 5), (9, 8), (9, 1), (10, 8), (10, 1), (11, 1), (11, 2);
```

[ Inline bearbeiten ] [ Bearbeiten ] [ PHP-Code erzeugen ]

✓ 10 Datensätze eingefügt.

ID der eingefügten Zeile: 10 (Die Abfrage dauerte 0,0002 Sekunden.)

```
-- Einfügen von 10 Beispiel-Leihanfragen INSERT INTO BorrowRequests (UserId,
BookId, PickupWindowStart, PickupWindowEnd, RequestedUntilTime, RequestTime,
BorrowRequestState) VALUES (2, 1, '2026-08-26 17:00:00', '2026-08-26 19:00:00',
'2026-08-30 23:59:59', '2026-08-25 09:00:00', 'Pending'), (3, 2, '2026-08-27
16:00:00', '2026-08-27 18:00:00', '2026-08-31 23:59:59', '2026-08-25 09:30:00',
'Accepted'), (4, 3, '2026-08-28 15:00:00', '2026-08-28 17:00:00', '2026-09-01
```

[ Bearbeiten ]

✓ 10 Datensätze eingefügt.

ID der eingefügten Zeile: 10 (Die Abfrage dauerte 0,0004 Sekunden.)

```
-- Einfügen von 10 Beispiel-Leihungen INSERT INTO Borrows (OwnerId, BorrowerId,
BookId, PickupLocation, PickupWindowStart, PickupWindowEnd, ReturnLocation,
ReturnWindowStart, ReturnWindowEnd, BorrowState) VALUES (1, 2, 1, 'Berlin
Hauptbahnhof', '2026-08-26 17:00:00', '2026-08-26 19:00:00', 'Berlin
Hauptbahnhof', '2026-09-20 17:00:00', '2026-09-20 19:00:00', 'Active'), (2, 3,
2, 'Hamburg Hauptbahnhof', '2026-08-27 16:00:00', '2026-08-27 18:00:00',
```

[ Bearbeiten ]

✓ 10 Datensätze eingefügt.

ID der eingefügten Zeile: 10 (Die Abfrage dauerte 0,0004 Sekunden.)

```
-- Einfügen von 10 Beispiel-Buchbewertungen INSERT INTO Valuations (BookId, UserId, Grade, Comment,
ValuationTime) VALUES (1, 2, 5, 'Sehr gutes Buch, liest sich gut weg ... guter Zustand.', '2026-08-20
10:00:00'), (2, 3, 4, 'Sehr interessant und gut geschrieben. Am Ende ein bisschen Längen...', '2026-08-21
11:00:00'), (3, 4, 5, 'Ein Klassiker :)'), '2026-08-21 14:00:00'), (4, 5, 5, 'Sehr empfehlenswert.', '2026-
08-22 09:30:00'), (5, 6, 4, 'Guter Zustand.', '2026-08-22 12:00:00'), (6, 7, 5, 'Sehr spannende Geschichte,
aber Charaktere etwas platt.', '2026-08-23 10:15:00'), (7, 8, 4, 'Guter Krimi.', '2026-08-23 13:00:00'),
```

[ Inline bearbeiten ] [ Bearbeiten ] [ PHP-Code erzeugen ]

✓ 10 Datensätze eingefügt.

ID der eingefügten Zeile: 10 (Die Abfrage dauerte 0,0004 Sekunden.)

```
-- Einfügen von 10 Beispiel-Beschwerden INSERT INTO Complaints (Complaint, BookId, ComplaintUserId,
ComplainerUserId, ComplaintTime) VALUES ('Der Typ kam viel zu spät!! Was soll das?', 1, 2, 1, '2026-08-20
09:00:00'), ('Buch war stärker beschädigt als behauptet.', 2, 3, 2, '2026-08-20 11:00:00'), ('Abholtermin
wurde überhaupt nicht eingehalten!', 3, 4, 3, '2026-08-21 10:30:00'), ('Rückgabe erfolgte viel zu spät!!!',
4, 5, 4, '2026-08-21 14:00:00'), ('Beschreibung des Buchzustands war nicht unbedingt ... vollständig ;)'),
5, 6, 5, '2026-08-22 09:45:00'), ('Übergabeort wurde seeehr kurzfristig geändert.', 6, 7, 6, '2026-08-22
```

[ Bearbeiten ]

# Test Case: User-Profil anlegen und ändern

```
-- User anlegen -> Registrierung
INSERT INTO Users (Email, Password, Name, Address, CityId, RoleId, RegistrationDate,
UserState)
VALUES ('joe.davola@example.com', 'password11', 'Joe Davola', 'Lindenstraße 15', 1,
1, NOW(), 'Active');

SET @new_user_id = LAST_INSERT_ID();

-- Userprofil ändern
UPDATE Users
SET Name = 'Crazy Joe Davola', Address = 'Sonnenallee 42', CityId = 2
WHERE UserId = @new_user_id;
```

1. Neuer User wird hinzugefügt
2. Zuletzt gesetzte Id wird als "User Id" für weitere Verwendung gespeichert
3. Userprofil des neuen Users wird geändert

✓ 1 Datensatz eingefügt.

ID der eingefügten Zeile: 11 (Die Abfrage dauerte 0,0007 Sekunden.)

```
-- ===== -- Test Cases für User: User-Verwaltung --
===== -- User anlegen -> Registrierung
INSERT INTO Users (Email,
Password, Name, Address, CityId, RoleId, RegistrationDate, UserState) VALUES ('joe.davola@example.com',
'password11', 'Joe Davola', 'Lindenstraße 15', 1, 1, NOW(), 'Active');
```

[ Inline bearbeiten ] [ Bearbeiten ] [ PHP-Code erzeugen ]

✓ MySQL lieferte ein leeres Resultat zurück (d.h. null Datensätze). (Die Abfrage dauerte 0,0001 Sekunden.)

```
SET @new_user_id = LAST_INSERT_ID();
```

[ Inline bearbeiten ] [ Bearbeiten ] [ PHP-Code erzeugen ]

✓ 1 Datensatz betroffen. (Die Abfrage dauerte 0,0004 Sekunden.)

```
-- Userprofil ändern
UPDATE Users SET Name = 'Crazy Joe Davola', Address = 'Sonnenallee 42', CityId = 2 WHERE
UserId = @new_user_id;
```

[ Inline bearbeiten ] [ Bearbeiten ] [ PHP-Code erzeugen ]

# Test Case: Buch anlegen, ändern, löschen

```
-- =====  
-- Test Cases für User: Bücherverwaltung  
-- =====  
  
-- Buch anlegen  
INSERT INTO Books (Name, Description, ISBN, PublisherId, LanguageId, MaxBorrowTime, OwnerId)  
VALUES ('Neuromancer', 'Sci-Fi Klassiker von William Gibson.', '9780441569564', 1, 2, 21, 2);  
  
SET @test_book_id = LAST_INSERT_ID();  
  
-- Genre und Autor verknüpfen  
INSERT INTO BookAuthors (BookId, AuthorId) VALUES (@test_book_id, 2);  
INSERT INTO BookGenres (BookId, GenreId) VALUES (@test_book_id, 2);  
  
-- Buch ändern  
UPDATE Books  
SET Description = 'Aktualisierte Beschreibung: Ein absoluter Cyberpunk-Klassiker.',  
MaxBorrowTime = 28  
WHERE BookId = @test_book_id;  
  
-- Buch löschen  
DELETE FROM Books  
WHERE BookId = @test_book_id;
```

1. Neues Buch wird hinzugefügt
2. Zuletzt gesetzte Id wird als Buch-Id für spätere Verwendung gespeichert
3. Buch-Autoren-Verknüpfung wird hinzugefügt
4. Buch-Genre-Verknüpfung wird hinzugefügt
5. Buch-Beschreibung wird geändert
6. Buch wird gelöscht

✓ 1 Datensatz eingefügt.  
ID der eingefügten Zeile: 12 (Die Abfrage dauerte 0,0003 Sekunden.)

```
-- ===== -- Test Cases für User:  
Bücherverwaltung -- Buch anlegen  
INSERT INTO Books (Name, Description, ISBN, PublisherId, LanguageId,  
MaxBorrowTime, OwnerId) VALUES ('Neuromancer', 'Sci-Fi Klassiker von William  
Gibson.', '9780441569564', 1, 2, 21, 2);
```

[ Inline bearbeiten ] [ Bearbeiten ] [ PHP-Code erzeugen ]

✓ MySQL lieferte ein leeres Resultat zurück (d.h. null Datensätze). (Die Abfrage dauerte 0,0001 Sekunden.)

```
SET @test_book_id = LAST_INSERT_ID();
```

[ Inline bearbeiten ] [ Bearbeiten ] [ PHP-Code erzeugen ]

✓ 1 Datensatz eingefügt.  
ID der eingefügten Zeile: 12 (Die Abfrage dauerte 0,0003 Sekunden.)

```
-- Genre und Autor verknüpfen INSERT INTO BookAuthors (BookId, AuthorId) VALUES  
(@test_book_id, 2);
```

[ Inline bearbeiten ] [ Bearbeiten ] [ PHP-Code erzeugen ]

✓ 1 Datensatz eingefügt.  
ID der eingefügten Zeile: 12 (Die Abfrage dauerte 0,0002 Sekunden.)

```
INSERT INTO BookGenres (BookId, GenreId) VALUES (@test_book_id, 2);
```

[ Inline bearbeiten ] [ Bearbeiten ] [ PHP-Code erzeugen ]

✓ 1 Datensatz betroffen. (Die Abfrage dauerte 0,0002 Sekunden.)

```
-- Buch ändern UPDATE Books SET Description = 'Aktualisierte Beschreibung: Ein  
absoluter Cyberpunk-Klassiker.', MaxBorrowTime = 28 WHERE BookId = @test_book_id;
```

[ Inline bearbeiten ] [ Bearbeiten ] [ PHP-Code erzeugen ]

✓ 1 Datensatz gelöscht. (Die Abfrage dauerte 0,0022 Sekunden.)

```
-- Buch löschen DELETE FROM Books WHERE BookId = @test_book_id;
```

[ Inline bearbeiten ] [ Bearbeiten ] [ PHP-Code erzeugen ]



# Test Case: Buch-Leihvorgang

```
-- Ausleihanfrage stellen (User 3 fragt Buch 4 bei User 4 an)
INSERT INTO BorrowRequests (UserId, BookId, PickupWindowStart,
PickupWindowEnd, RequestedUntilTime, RequestTime, BorrowRequestState)
VALUES (3, 4, '2026-09-01 10:00:00', '2026-09-01 12:00:00', '2026-09-15
23:59:59', NOW(), 'Pending');
```

```
SET @borrow_request_id = LAST_INSERT_ID();
```

```
-- Status der Anfrage auf 'Accepted' setzen
```

```
UPDATE BorrowRequests
SET BorrowRequestState = 'Accepted'
WHERE BorrowRequestId = @borrow_request_id;
```

```
-- Buch tatsächlich ausleihen (Eintrag in Borrows-Tabelle erstellen)
```

```
INSERT INTO Borrows (OwnerId, BorrowerId, BookId, PickupLocation,
PickupWindowStart, PickupWindowEnd, BorrowState)
VALUES (4, 3, 4, 'Köln Dom', '2026-09-01 10:00:00', '2026-09-01
12:00:00', 'Active');
```

```
SET @active_borrow_id = LAST_INSERT_ID();
```

```
-- Buch zurückgeben
```

```
UPDATE Borrows
SET ReturnLocation = 'Köln Dom',
ReturnWindowStart = '2026-09-15 10:00:00',
ReturnWindowEnd = '2026-09-15 12:00:00',
BorrowState = 'Returned'
WHERE BorrowId = @active_borrow_id;
```

1. Leihanfrage wird gestellt
2. Zuletzt gesetzte Id wird als "BorrowRequestId" für spätere Verwendung gespeichert
3. Leihanfrage wird akzeptiert
4. Buch wird ausgeliehen
5. Zuletzt gesetzte Id wird als "BorrowId" für spätere Verwendung gespeichert
6. Buch wird zurückgegeben

✓ 1 Datensatz eingefügt  
ID der eingefügten Zeile: 11 (Die Abfrage dauerte 0,0004 Sekunden.)

```
-- ===== Test Cases für User: Bücher
ausleihen -- ===== -- Ausleihanfrage
stellen (User 3 fragt Buch 4 bei User 4 an) INSERT INTO BorrowRequests (UserId,
BookId, PickupWindowStart, PickupWindowEnd, RequestedUntilTime, RequestTime,
BorrowRequestState) VALUES (3, 4, '2026-09-01 10:00:00', '2026-09-01 12:00:00',
'2026-09-15 23:59:59', NOW(), 'Pending');
```

[ Inline bearbeiten ] [ Bearbeiten ] [ PHP-Code erzeugen ]

✓ MySQL lieferte ein leeres Resultat zurück (d.h. null Datensätze). (Die Abfrage dauerte 0,0001 Sekunden.)

```
SET @borrow_request_id = LAST_INSERT_ID();
```

[ Inline bearbeiten ] [ Bearbeiten ] [ PHP-Code erzeugen ]

✓ 1 Datensatz betroffen. (Die Abfrage dauerte 0,0002 Sekunden.)

```
-- Status der Anfrage auf 'Accepted' setzen UPDATE BorrowRequests SET
BorrowRequestState = 'Accepted' WHERE BorrowRequestId = @borrow_request_id;
```

[ Inline bearbeiten ] [ Bearbeiten ] [ PHP-Code erzeugen ]

✓ 1 Datensatz eingefügt.  
ID der eingefügten Zeile: 11 (Die Abfrage dauerte 0,0002 Sekunden.)

```
-- Buch tatsächlich ausleihen (Eintrag in Borrows-Tabelle erstellen) INSERT INTO
Borrows (OwnerId, BorrowerId, BookId, PickupLocation, PickupWindowStart,
PickupWindowEnd, BorrowState) VALUES (4, 3, 4, 'Köln Dom', '2026-09-01 10:00:00',
'2026-09-01 12:00:00', 'Active');
```

[ Inline bearbeiten ] [ Bearbeiten ] [ PHP-Code erzeugen ]

✓ MySQL lieferte ein leeres Resultat zurück (d.h. null Datensätze). (Die Abfrage dauerte 0,0001 Sekunden.)

```
SET @active_borrow_id = LAST_INSERT_ID();
```

[ Inline bearbeiten ] [ Bearbeiten ] [ PHP-Code erzeugen ]

✓ 1 Datensatz betroffen. (Die Abfrage dauerte 0,0004 Sekunden.)

```
-- Buch zurückgeben UPDATE Borrows SET ReturnLocation = 'Köln Dom',
ReturnWindowStart = '2026-09-15 10:00:00', ReturnWindowEnd = '2026-09-15
12:00:00', BorrowState = 'Returned' WHERE BorrowId = @active_borrow_id;
```

[ Inline bearbeiten ] [ Bearbeiten ] [ PHP-Code erzeugen ]

# Test Case: Bewertung und Beschwerden

```
-- Buch bewerten
INSERT INTO Valuations (BookId, UserId, Grade, Comment, ValuationTime)
VALUES (4, 3, 5, 'Super Abwicklung und genialer Roman!', NOW());

-- Beschwerden abgeben
INSERT INTO Complaints (Complaint, BookId, ComplainerUserId,
ComplainerUserId, ComplaintTime)
VALUES ('Das Buch hatte Flecken auf Seite 20.', 4, 4, 3, NOW());
```

1. Buch wird bewertet
2. Beschwerde wird abgegeben

✓ 1 Datensatz eingefügt.

ID der eingefügten Zeile: 11 (Die Abfrage dauerte 0,0002 Sekunden.)

```
-- Buch bewerten INSERT INTO Valuations (BookId, UserId, Grade, Comment,
ValuationTime) VALUES (4, 3, 5, 'Super Abwicklung und genialer Roman!', NOW());
```

[ Inline bearbeiten ] [ Bearbeiten ] [ PHP-Code erzeugen ]

✓ 1 Datensatz eingefügt.

ID der eingefügten Zeile: 11 (Die Abfrage dauerte 0,0002 Sekunden.)

```
-- Beschwerden abgeben INSERT INTO Complaints (Complaint, BookId,
ComplainerUserId, ComplainerUserId, ComplaintTime) VALUES ('Das Buch hatte Flecken
auf Seite 20.', 4, 4, 3, NOW());
```

[ Inline bearbeiten ] [ Bearbeiten ] [ PHP-Code erzeugen ]



# Test Case: Bücher nach Name, Autor, Genre suchen

-- Suche nach Buchtitel, Genre oder Autor (z. B. nach "Fantasy" oder "Tolkien")

```
SELECT DISTINCT
  b.BookId,
  b.Name AS BookTitle,
  a.Author,
  g.Genre,
  l.Language,
  u.Name AS Owner
FROM Books b
LEFT JOIN BookAuthors ba ON b.BookId = ba.BookId
LEFT JOIN Authors a ON ba.AuthorId = a.AuthorId
LEFT JOIN BookGenres bg ON b.BookId = bg.BookId
LEFT JOIN Genres g ON bg.GenreId = g.GenreId
LEFT JOIN Languages l ON b.LanguageId = l.LanguageId
LEFT JOIN Users u ON b.OwnerId = u.UserId
WHERE
  u.UserState = 'Active'
  AND (
    b.Name LIKE '%Herr der Ringe%'
    OR a.Author LIKE '%Tolkien%'
    OR g.Genre LIKE '%Fantasy%'
  );
```

Suche nach Büchern, deren Name "Herr der Ringe" enthält, deren Autor "Tolkien" enthält, oder deren Genre "Fantasy" enthält.

Der Buchbesitzer muss ein aktives (nicht gesperrtes) Userprofil aufweisen.

Beispiel-Testcase enthält Doppelseinträge (mehrere Autoren). Lösung siehe Testcase "Bücher mit allen Autoren und Genres anzeigen"

✓ Zeige Datensätze 0 - 3 (4 insgesamt, Die Abfrage dauerte 0,0005 Sekunden.)

```
-- Suche nach Buchtitel, Genre oder Autor (z. B. nach "Fantasy" oder "Tolkien") SELECT DISTINCT b.BookId,
b.Name AS BookTitle, a.Author, g.Genre, l.Language, u.Name AS Owner FROM Books b LEFT JOIN BookAuthors ba
ON b.BookId = ba.BookId LEFT JOIN Authors a ON ba.AuthorId = a.AuthorId LEFT JOIN BookGenres bg ON b.BookId
= bg.BookId LEFT JOIN Genres g ON bg.GenreId = g.GenreId LEFT JOIN Languages l ON b.LanguageId =
l.LanguageId LEFT JOIN Users u ON b.OwnerId = u.UserId WHERE u.UserState = 'Active' AND ( b.Name LIKE
'Herr der Ringe%' OR a.Author LIKE '%Tolkien%' OR g.Genre LIKE '%Fantasy%' );
```

[ Inline bearbeiten ] [ Bearbeiten ] [ PHP-Code erzeugen ]

☐ Alles anzeigen | Anzahl der Datensätze: 25 | Zeilen filtern: Diese Tabelle durchsuchen | Nach Schlüssel sortieren:

Zusätzliche Optionen

| BookId | BookTitle          | Author          | Genre   | Language | Owner          |
|--------|--------------------|-----------------|---------|----------|----------------|
| 3      | Der Herr der Ringe | J.R.R. Tolkien  | Fantasy | Englisch | Kel Varnsen    |
| 10     | Wachen! Wachen!    | Terry Pratchett | Fantasy | Englisch | Jerry Seinfeld |
| 11     | Der lange Krieg    | Terry Pratchett | Fantasy | Englisch | J. Peterman    |
| 11     | Der lange Krieg    | Stephen Baxter  | Fantasy | Englisch | J. Peterman    |

# Test Case: Bücher mit allen Autoren und Genres anzeigen

```
-- zeige alle Bücher
-- (mehrere Autoren und Genres werden in einer kommaseparierten Liste zusammengefasst)
SELECT
    b.Name AS Book,
    GROUP_CONCAT(DISTINCT a.Author ORDER BY a.Author SEPARATOR ', ') AS Authors,
    GROUP_CONCAT(DISTINCT g.Genre ORDER BY g.Genre SEPARATOR ', ') AS Genres
FROM Books b
LEFT JOIN BookAuthors ba
    ON b.BookId = ba.BookId
LEFT JOIN Authors a
    ON ba.AuthorId = a.AuthorId
LEFT JOIN BookGenres bg
    ON b.BookId = bg.BookId
LEFT JOIN Genres g
    ON bg.GenreId = g.GenreId
GROUP BY
    b.BookId,
    b.Name;
```

Anzeigen aller Bücher, dabei werden Autoren und Genres jeweils alphabetisch sortiert in einer einzigen Spalte dargestellt, komma-separiert.

✓ Zeige Datensätze 0 - 11 (12 insgesamt, Die Abfrage dauerte 0,0005 Sekunden.)

```
-- zeige alle Bücher -- (mehrere Autoren und Genres werden in einer kommaseparierten Liste zusammengefasst)
SELECT b.Name AS Book, GROUP_CONCAT(DISTINCT a.Author ORDER BY a.Author SEPARATOR ', ') AS Authors,
GROUP_CONCAT(DISTINCT g.Genre ORDER BY g.Genre SEPARATOR ', ') AS Genres FROM Books b LEFT JOIN BookAuthors
ba ON b.BookId = ba.BookId LEFT JOIN Authors a ON ba.AuthorId = a.AuthorId LEFT JOIN BookGenres bg ON
b.BookId = bg.BookId LEFT JOIN Genres g ON bg.GenreId = g.GenreId GROUP BY b.BookId, b.Name;
```

[ [Inline bearbeiten](#) ] [ [Bearbeiten](#) ] [ [PHP-Code erzeugen](#) ]

☐ Alles anzeigen

Anzahl der Datensätze: 25

Zeilen filtern:

Nach Schlüssel sortieren:

Zusätzliche Optionen

| Book                           | Authors                         | Genres                   |
|--------------------------------|---------------------------------|--------------------------|
| Der Bluthund                   | Lee Child                       | Krimi                    |
| 1984                           | George Orwell                   | Roman, Science-Fiction   |
| Der Herr der Ringe             | J.R.R. Tolkien                  | Fantasy                  |
| Dune                           | Frank Herbert                   | Science-Fiction          |
| Die Drei Sonnen                | Cixin Liu                       | Science-Fiction          |
| Foundation                     | Isaac Asimov                    | Science-Fiction          |
| Weißwurst-Connection           | Rita Falk                       | Krimi                    |
| Es                             | Stephen King                    | Horror                   |
| Per Anhalter durch die Galaxis | Douglas Adams                   | Komödie, Science-Fiction |
| Wachen! Wachen!                | Terry Pratchett                 | Fantasy, Komödie         |
| Der lange Krieg                | Stephen Baxter, Terry Pratchett | Fantasy, Science-Fiction |
| Tagebuch eines Killerbots      | Rita Falk                       | Komödie, Science-Fiction |

# Test Case: verliehene Bücher anzeigen

```
-- Zeige alle gerade verliehenen Bücher.  
SELECT  
  Books.Name AS Book,  
  Owner.Name AS Owner,  
  Borrower.Name AS Borrower,  
  Borrows.BorrowState  
FROM Borrows  
JOIN Books  
  ON Borrows.BookId = Books.BookId  
JOIN Users AS Owner  
  ON Borrows.OwnerId = Owner.UserId  
JOIN Users AS Borrower  
  ON Borrows.BorrowerId = Borrower.UserId  
WHERE Borrows.BorrowState = 'Active';
```

Suche nach gerade verliehenen Büchern (BorrowState = "Active").

✓ Zeige Datensätze 0 - 6 (7 insgesamt, Die Abfrage dauerte 0,0005 Sekunden.)

```
-- Zeige alle gerade verliehenen Bücher. SELECT Books.Name AS Book, Owner.Name AS Owner, Borrower.Name AS  
Borrower, Borrows.BorrowState FROM Borrows JOIN Books ON Borrows.BookId = Books.BookId JOIN Users AS Owner ON  
Borrows.OwnerId = Owner.UserId JOIN Users AS Borrower ON Borrows.BorrowerId = Borrower.UserId WHERE  
Borrows.BorrowState = 'Active';
```

[ Inline bearbeiten ] [ Bearbeiten ] [ PHP-Code erzeugen ]

☐ Alles anzeigen

Anzahl der Datensätze:

25

Zeilen filtern:

Diese Tabelle durchsuchen

Nach Schlüssel sortieren:

Zusätzliche Optionen

| Book                           | Owner                      | Borrower                   | BorrowState |
|--------------------------------|----------------------------|----------------------------|-------------|
| Der Bluthund                   | Art Vandelay               | H.E. Pennypacker           | Active      |
| Der Herr der Ringe             | Kel Varnsen                | George Costanza            | Active      |
| Dune                           | George Costanza            | Elaine Benes               | Active      |
| Foundation                     | Cosmo Kramer               | Franklin Delano Romanowsky | Active      |
| Weißwurst-Connection           | Franklin Delano Romanowsky | Jerry Seinfeld             | Active      |
| Es                             | Jerry Seinfeld             | J. Peterman                | Active      |
| Per Anhalter durch die Galaxis | J. Peterman                | Kenny Bania                | Active      |

# Test Case: Admin - User sperren

```
-- User durch Admin sperren
UPDATE Users
SET UserState = 'Banned'
WHERE UserId = @new_user_id;

-- alle Anfragen des gesperrten Users auf Bücher abbrechen
UPDATE BorrowRequests
SET BorrowRequestState = 'Cancelled'
WHERE UserId = @new_user_id
AND BorrowRequestState = 'Pending';

-- alle Anfragen auf Bücher des gesperrten Users abbrechen
UPDATE BorrowRequests br
JOIN Books b
ON br.BookId = b.BookId
SET br.BorrowRequestState = 'Cancelled'
WHERE b.OwnerId = @new_user_id
AND br.BorrowRequestState = 'Pending';

-- gesperrten User anzeigen
SELECT
    UserId,
    Name,
    RoleId,
    UserState
FROM Users
WHERE UserId = @new_user_id;
```

1. "UserState" wird auf 'Banned' gesetzt
2. Laufende Leihanfragen des Users werden abgebrochen
3. Laufende Leihanfragen an den User werden abgebrochen
4. Gesperrter User wird angezeigt

✓ 1 Datensatz betroffen. (Die Abfrage dauerte 0,0003 Sekunden.)

```
-- ===== Test Cases für Admin: User sperren =====
-- User durch Admin sperren UPDATE Users SET UserState = 'Banned'
WHERE UserId = @new_user_id;
```

[ Inline bearbeiten ] [ Bearbeiten ] [ PHP-Code erzeugen ]

✓ 0 Datensätze betroffen. (Die Abfrage dauerte 0,0002 Sekunden.)

```
-- alle Anfragen des gesperrten Users auf Bücher abbrechen UPDATE BorrowRequests SET BorrowRequestState =
'Cancelled' WHERE UserId = @new_user_id AND BorrowRequestState = 'Pending';
```

[ Inline bearbeiten ] [ Bearbeiten ] [ PHP-Code erzeugen ]

✓ 0 Datensätze betroffen. (Die Abfrage dauerte 0,0004 Sekunden.)

```
-- alle Anfragen auf Bücher des gesperrten Users abbrechen UPDATE BorrowRequests br JOIN Books b ON br.BookId
= b.BookId SET br.BorrowRequestState = 'Cancelled' WHERE b.OwnerId = @new_user_id AND br.BorrowRequestState =
'Pending';
```

[ Inline bearbeiten ] [ Bearbeiten ] [ PHP-Code erzeugen ]

✓ Zeige Datensätze 0 - 0 (1 insgesamt, Die Abfrage dauerte 0,0001 Sekunden.)

```
-- gesperrten User anzeigen SELECT UserId, Name, RoleId, UserState FROM Users WHERE UserId = @new_user_id;
```

[ Inline bearbeiten ] [ Bearbeiten ] [ PHP-Code erzeugen ]

☐ Alles anzeigen | Anzahl der Datensätze: 25 | Zeilen filtern: Diese Tabelle durchsuchen

Zusätzliche Optionen

|                                                                                                        | UserId | Name             | RoleId | UserState |
|--------------------------------------------------------------------------------------------------------|--------|------------------|--------|-----------|
| <input type="checkbox"/> Bearbeiten <input type="checkbox"/> Kopieren <input type="checkbox"/> Löschen | 11     | Crazy Joe Davola |        | 1 Banned  |

# Test Case: Admin - User und Bücher löschen

```
-- User durch Admin komplett löschen
-- (nur wenn keine Borrows existieren - BorrowRequests werden
automatisch gelöscht)
DELETE FROM Users
WHERE UserId = @new_user_id;

-- Buch löschen
DELETE FROM Books
WHERE BookId = 8;
```

1. Löschen eines Users → ist nur möglich, wenn keine offenen Leihungen und Leihgaben des Users existieren d.h. die Historie des Users bleibt erhalten
2. Löschen eines Buchs

✓ 1 Datensatz gelöscht. (Die Abfrage dauerte 0,0020 Sekunden.)

```
-- User durch Admin komplett löschen -- (nur wenn keine Borrows existieren - BorrowRequests werden
automatisch gelöscht) DELETE FROM Users WHERE UserId = @new_user_id;
```

[ Inline bearbeiten ] [ Bearbeiten ] [ PHP-Code erzeugen ]

✓ 1 Datensatz gelöscht. (Die Abfrage dauerte 0,0002 Sekunden.)

```
-- Buch löschen DELETE FROM Books WHERE BookId = 8;
```

[ Inline bearbeiten ] [ Bearbeiten ] [ PHP-Code erzeugen ]

# Test Case: Admin - Bücher aller aktiven User suchen

```
-- Zeige alle Bücher von allen Usern
SELECT
  Books.Name AS Book,
  Users.Name AS Owner
FROM Books
JOIN Users
  ON Books.OwnerId = Users.UserId
WHERE Users.UserState = 'Active';
```

Anzeigen aller Bücher mit ihren jeweiligen Besitzern → User muss 'Active' sein, darf nicht gesperrt sein.

✓ Zeige Datensätze 0 - 11 (12 insgesamt, Die Abfrage dauerte 0,0003 Sekunden.)

```
-- Zeige alle Bücher von allen Usern
SELECT Books.Name AS Book, Users.Name AS Owner
FROM Books JOIN Users
  ON Books.OwnerId = Users.UserId
WHERE Users.UserState = 'Active';
```

[ Inline bearbeiten ] [ Bearbeiten ] [ PHP-Code erzeugen ]

☐ Alles anzeigen | Anzahl der Datensätze: 25 | Zeilen filtern: Diese Tabelle durchsuchen | Nach Schlüssel sortieren:

Zusätzliche Optionen

| Book                           | Owner                      |
|--------------------------------|----------------------------|
| Der Bluthund                   | Art Vandelay               |
| 1984                           | H.E. Pennypacker           |
| Der Herr der Ringe             | Kel Varnsen                |
| Dune                           | George Costanza            |
| Die Drei Sonnen                | Elaine Benes               |
| Foundation                     | Cosmo Kramer               |
| Weißwurst-Connection           | Franklin Delano Romanowsky |
| Es                             | Franklin Delano Romanowsky |
| Per Anhalter durch die Galaxis | Jerry Seinfeld             |
| Wachen! Wachen!                | Jerry Seinfeld             |
| Der lange Krieg                | J. Peterman                |
| Tagebuch eines Killerbots      | Kenny Bania                |

# Zusammenfassung der Implementation

|                                   |                            |
|-----------------------------------|----------------------------|
| Datenbank-Managementsystem (DBMS) | MySQL / XAMPP / phpMyAdmin |
| Code Editor                       | Sublime Text               |
| Ausgabedatei (inkl. aller Tests)  | Buchtausch_Setup.sql       |
| Lines of Code (inkl. Kommentaren) | 701                        |
| Anzahl Tabellen                   | 14                         |